

Installing R, Python, PostgreSQL, Quarto and Positron on macOS

Big Data Algorithms — setup guide

These steps install the current versions of the software we use in the course on a Mac. They work on both Apple Silicon (M-series) and Intel Macs. Ubuntu users, including Ubuntu inside WSL on Windows, should follow “software_setup_ubuntu.qmd” instead.

We install everything with Homebrew, the standard package manager for macOS. Run every command in the Terminal app. As of October 2026, you should end up with:

Software	Version	Check with
R	4.6	<code>R --version</code>
Python	3.14	<code>python3 --version</code>
PostgreSQL	18	<code>psql --version</code>
Quarto	1.10	<code>quarto --version</code>
Positron	2026.09	<code>Positron > About Positron</code>

Homebrew

Check whether you already have Homebrew:

```
brew --version
```

If the command is not found, install Homebrew:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

It asks for your Mac password. When the installer finishes, it prints a short “Next steps” section with commands that add `brew` to your PATH. Run them, then open a new terminal window.

Make sure Homebrew knows about the latest versions before installing anything:

```
brew update
```

R

```
brew install --cask r-app
```

This downloads the official installer from CRAN, the home of R, and runs it. It asks for your Mac password. Note the `--cask`: do not use plain `brew install r`. That installs a different build of R that cannot use CRAN's ready-made packages, so every package you install would have to be compiled on your Mac.

Check the installation:

```
R --version
```

The first line should begin with `R version 4.6`.

Python

```
brew install python  
python3 --version
```

This should print `Python 3.14` followed by a third number. Note that the command is `python3`, not `python`.

macOS comes with an older Python of its own, so make sure the terminal uses the one from Homebrew:

```
which python3
```

This should print `/opt/homebrew/bin/python3` (Apple Silicon) or `/usr/local/bin/python3` (Intel). If it prints `/usr/bin/python3`, open a new terminal window and try again.

Homebrew does not allow `pip` to install packages into its Python (you would get an `externally-managed-environment` error). Instead, create a virtual environment for the course and install packages there:

```
python3 -m venv ~/.venvs/bigdata
source ~/.venvs/bigdata/bin/activate
```

Your prompt now begins with **(bigdata)**, and inside the environment the commands **python** and **pip** refer to the environment's own copies. Type **deactivate** to leave it. You need to run the **source** line again in every new terminal where you want to use the environment. Positron looks for environments in **~/.venvs**, so it will find this one on its own.

PostgreSQL

Install PostgreSQL 18 and start the database server:

```
brew install postgresql@18
brew services start postgresql@18
```

brew services also restarts the server every time you log in. Homebrew does not put the PostgreSQL commands on your **PATH**, so add them yourself:

```
echo "export PATH=\"\$(brew --prefix postgresql@18)/bin:\$PATH\"" >> ~/.zshrc
source ~/.zshrc
```

Homebrew sets up PostgreSQL so that your Mac username is the database administrator. Create a database with the same name as your username:

```
createdb $USER
```

From now on **psql** connects without any extra options:

```
psql -c "SELECT version();" 
```

This should print one row beginning with PostgreSQL 18. Typing **psql** alone opens the interactive SQL prompt and **\q** quits it.

If **psql** cannot connect, check that the server is running with **brew services list**. The line for **postgresql@18** should say **started**.

Quarto

```
brew install --cask quarto
```

It asks for your Mac password. Check the installation:

```
quarto --version  
quarto check
```

`quarto check` lists everything Quarto can find on your computer, and should show the R and Python you just installed. It may also tell you that Jupyter (for Python) and the `knitr` package (for R) are not installed yet. The next section installs `knitr`, and Jupyter is not needed for the setup in this guide.

R and Python together in Quarto

A single Quarto document can contain both R and Python code. Quarto runs the code in such a document with R's `knitr` and `rmarkdown` packages, and the `reticulate` package lets R start Python for the Python parts. Install all three:

```
Rscript -e 'install.packages(c("knitr", "rmarkdown", "reticulate"), repos = "https://cloud.r-project.org")
```

Left alone, `reticulate` chooses a Python by itself, and it may not be your `bigdata` environment. Packages you installed there would then appear to be missing when you render a document. Tell `reticulate` which Python to use by setting the variable `RETICULATE_PYTHON` in `~/.Renviron`, a file R reads every time it starts:

```
echo "RETICULATE_PYTHON=$HOME/.venvs/bigdata/bin/python" >> ~/.Renviron
```

Check the result:

```
Rscript -e 'reticulate::py_config()'
```

The first line should read `python:` followed by the path of `.venvs/bigdata/bin/python` inside your home directory. A line saying `numpy: [NOT FOUND]` is expected, because nothing has been installed into the environment yet.

Positron

Positron is the editor (IDE) we use for R, Python and Quarto.

```
brew install --cask positron
```

Open Positron from your Applications folder. To be able to start it from the terminal as well, press Cmd+Shift+P inside Positron to bring up the Command Palette, type “Install ‘positron’ command in PATH” and run that command. After you restart the terminal, `positron .` opens the current folder in Positron.

Check that Positron sees R and Python

Click the interpreter selector in the top-right corner of the Positron window. It should list R 4.6 and Python 3.14, including the `bigdata` environment. Select R, and the Console at the bottom should show R’s startup message. Then do the same for Python.

Positron comes with its own copy of Quarto. We installed Quarto separately so that the `quarto` command also works in the terminal.

Updating later

Homebrew updates everything you installed with it:

```
brew update && brew upgrade
```

Positron also offers updates by itself when a new version is released.